

CLC – Milestone 3

SWE-410

PhytoPi

Daniel Grijalva, Nolan Tuttle

Bill Hughes

03/29/26

Version and Team Member Page

Version 2.0

Revision Date: 03/29/26

Print Name	Signature	Date
Nolan Tuttle	Nolan Tuttle	3/29/26
Daniel Grijalva	Daniel Grijalva	3/29/26

Main Body

1. GitHub Repository Link

<https://github.com/nolantuttle/PhytoPi-ESP32>

2. Video Presentation Link

<https://youtu.be/YEbxDFSi2nA>

3. Project Overview

Small IoT sensor node on the ESP32 that: Reads humidity and temperature from a DHT11 wired to the board. Connects to LAN over Wi-Fi and to a MQTT broker (on a Raspberry Pi 5). Publishes those readings to MQTT topics under something like `gcu/swe410/dht11/...` so another system can consume them. Shows it works by subscribing on the Pi (e.g. with `mosquitto_sub`) and capturing that output for evidence.

4. System architecture

- a. **ESP32** — Runs the firmware brings up non-volatile storage, connects to Wi-Fi as a station, maintains an MQTT client to a broker, reads the DHT11 on a configured GPIO, and periodically publishes telemetry. Serial logging is used for local observation.
- b. **DHT11** — Ambient temperature and relative humidity sensor. The firmware reads it through a small wrapper around `esp-idf-lib DHT` code; readings are logged in a fixed format and encoded as JSON for MQTT.
- c. **Wi-Fi** — Provides the transport path from the ESP32 to the LAN so the device can reach the MQTT broker (typically the same subnet as a home/lab access point).
- d. **MQTT broker** — Accepts authenticated client connections and routes messages by topic. In this project, the broker URI and credentials are configurable via `Kconfig (mqtt_manager)`; the root README shows subscribing on the broker host with a wildcard under `gcu/swe410/dht11/#`, which implies the broker runs on a machine reachable at that address (often the Raspberry Pi when it is the broker host).
- e. **Raspberry Pi 5** — Role as described in the project README: aggregate and process plant-related data and support visualization. For the MQTT milestone as documented here, the Pi is the natural place to run Mosquitto (or another broker) and tools such as `mosquitto_sub` to verify publishes.

5. Code refactoring summary

- a. **Separation of concerns** — Connectivity is split into `wifi_manager` (STA, events, wait-for-IP) and `mqtt_manager` (client init, events, subscribe on connect, `mqtt_publish` helper). Sensor access is isolated in `dht11_wrapper` instead of mixing DHT driver calls into `main.c`.
- b. **Configuration** — SSID/password and broker URI/username/password live in `Kconfig (wifi_manager, mqtt_manager)`, similar to a maintainable lab project rather than hard-coded literals in one file.
- c. **Naming** — The custom component is `mqtt_manager`, which avoids overloading the name `mqtt_client` (reserved for ESP-IDF's client API).

- d. **Application flow** — app_main follows a clear sequence: NVS → Wi-Fi → MQTT start → DHT init → loop with delay between DHT reads (comment notes DHT11 timing constraints).
- e. **Telemetry format** — Comments in main.c state that serial logs match the lab format and MQTT JSON carries humidity and temperature_c only (Fahrenheit can be derived downstream).

6. MQTT topic design

Topic	Direction (relative to ESP32)	Payload
gcu/swe410/dht11/telemetry	Publish (QoS 1, no retain)	JSON string: {"humidity":<float>,"temperature_c":<float>}
gcu/swe410/dht11/commands	Subscribe (QoS 1 on subscribe)	Any payload the broker delivers; the firmware logs topic and data only no command parsing or actuation in the current code.

7. Implementation explanation

- a. main/main.c — Orchestrates startup and the infinite telemetry loop: NVS init, wifi_init_sta + wifi_wait_connected, mqtt_app_start, dht11_init, then read/publish every 2 seconds with error handling and logging.
- b. wifi_manager — Default STA setup, event handlers for connect/disconnect/GOT_IP, reconnect on disconnect, blocking wait until an IP is assigned. Sets Wi-Fi power save to none to reduce timing jitter for the bit-banged DHT protocol.
- c. mqtt_manager — Builds esp_mqtt_client_config_t from Kconfig, registers an event handler, subscribes to gcu/swe410/dht11/commands on MQTT_EVENT_CONNECTED, exposes mqtt_publish for string payloads.
- d. dht11_wrapper — Wraps dht_read_float_data for DHT11, configures GPIO pull-up, fills dht11_reading_t (humidity, °C, °F).

8. Build / flash / run procedure

- a. Load the ESP-IDF environment (example: source the export.sh for the installed IDF version).
- b. Change to the project directory: PhytoPiESP32 under this repo.
- c. Configure the project (idf.py menuconfig) so Wi-Fi and MQTT Kconfig values match the lab network and broker.
- d. Build, flash, and monitor serial output, e.g. idf.py -p <serial port> -b 115200 flash monitor (the README uses /dev/ttyUSB0 as an example).
- e. On the machine running the broker (often the Pi), run a subscriber such as mosquitto_sub against 127.0.0.1 with the appropriate topic filter and credentials as in the README.

9. Testing and verification

- a. Publishing — Confirmed by running a subscriber on the broker host that matches `gcu/swe410/dht11/#` (or `.../telemetry`) and observing JSON messages when the ESP32 is flashed and connected.
- b. Subscription — The firmware subscribes to `gcu/swe410/dht11/commands` on connect; confirmation appears in the serial log (`MQTT_EVENT_SUBSCRIBED`, “Subscribed to ...”). Inbound messages are printed under `MQTT_EVENT_DATA`. End-to-end subscription testing would consist of publishing that topic from another client and checking the ESP32 serial log for the received topic and payload.

10. Issues encountered and resolutions

- a. The repo does not include a dedicated debugging log. Issues that are reflected in code or comments:
 - i. DHT reliability vs Wi-Fi — Comment in `wifi_manager.c`: power save is disabled (`WIFI_PS_NONE`) to limit modem sleep jitter because DHT timing is sensitive while STA is active. The DHT wrapper comment notes external pull-up may still be needed for reliable edges with Wi-Fi active.
 - ii. Read failures — `main.c` logs a warning and retries after 2 s if `dht11_read` fails.
 - iii. MQTT errors — `mqtt_manager` logs TLS/transport error fields when `MQTT_EVENT_ERROR` occurs.

ESP32 / VS Code Evidence

- Project folder structure
- main.c
- DHT11 wrapper files
- Successful build output
- Successful flash / monitor output

```

CMakeLists.txt
README.md
# Version Control and Workflow
Development uses Git and GitHub for version control
Feature branches are used for new functionality, bug fixes, and experimental changes
Pull requests are created for merging changes into main after review
Releases are tagged using Semantic Versioning (MAJOR.MINOR.PATCH)
# System Sensors
Capacitive Soil Moisture Sensor: Determines soil water content and triggers watering decisions
Temperature and Humidity Sensor: Monitors ambient conditions affecting plant growth
ESP32 Data Collection: ESP32 reads all sensors and transmits data via BLE to the Raspberry Pi
# Read client messages
pi: mosquitto_sub -h 127.0.0.1 -t 'gcu/swe410/dht11/#' -u mqttuser -P 'DanielPassword' -v
pc: ~/.esp/v5.5.2/esp-idf/export.sh
cd /home/danielg/Documents/Projects/School/PhytoPi_Esp32/PhytoPi-ESP32/PhytoPiEsp32
idf.py -p - /dev/ttyUSB0 -b 115200 flash monitor
Problems Output Debug Console Terminal Ports ESP-IDF
I (4249) wifi:dp: 1, bi: 102400, li: 3, scale listen interval from 307200 us to 307200 us
I (4289) wifi:-ba-add-idx:0 (ifx:0, d8:3a:dd:d2:cd:11), tid:7, ssn:0, winSize:64
I (4329) wifi:dp: 2, bi: 102400, li: 4, scale listen interval from 307200 us to 409600 us
I (4329) wifi:AP's beacon interval = 102400 us, DTIM period = 2
I (5309) esp_netif_handlers: sta ip: 192.168.50.17, mask: 255.255.255.0, gw: 192.168.50.1
I (5309) WIFI_MANAGER: Got IP: 192.168.50.17
I (5309) MQTT_MANAGER: Connecting to broker: mqtt://192.168.50.1:1883
I (5319) DHT11_WRAPPER: Initialized DHT11 on GPIO 4
I (5319) MAIN: Starting DHT11 telemetry loop (GPIO 4)
I (5349) MAIN: Humidity: 35.0%, Temp: 77.0F (25.0C)
I (5359) wifi:-ba-add-idx:1 (ifx:0, d8:3a:dd:d2:cd:11), tid:0, ssn:3, winSize:64
I (5379) MQTT_MANAGER: MQTT connected to broker
I (5379) MQTT_MANAGER: Subscribed to gcu/swe410/dht11/commands
I (5389) MQTT_MANAGER: MQTT subscribe confirmed msg_id=39440
I (7399) MAIN: Humidity: 35.0%, Temp: 77.0F (25.0C)
I (9419) MAIN: Humidity: 35.0%, Temp: 77.0F (25.0C)
I (11439) MAIN: Humidity: 35.0%, Temp: 77.0F (25.0C)
I (13459) MAIN: Humidity: 35.0%, Temp: 77.0F (25.0C)

```

Raspberry Pi 5 Evidence

- Mosquitto subscriber command
- Live terminal output showing humidity and temperature topics

[illegible]

GitHub Evidence

- Repository main page
- Commit history
- Docs folder
- README

The screenshot displays the GitHub repository page for **PhytoPi-ESP32** by user **nolanruttle**. The repository is public and has 2 branches and 0 tags. The commit history table shows the following entries:

Commit	Message	Time
Daniel G	Milestone 3: DHT11 readings and MQTT telemetry via modular components	1 hour ago
PhytoPi-ESP32	Milestone 3: DHT11 readings and MQTT telemetry via mo...	1 hour ago
components	Update file structure	last month
gitignore	Initial commit	2 months ago
LICENSE	Initial commit	2 months ago
README.md	Change data transmission method to Wi-Fi	last month

The README content is as follows:

PhytoPi-ESP32

PhytoPi is an IoT-based controlled environment system for supporting a plant throughout its entire growth cycle with minimal human intervention. This repository tracks our version of this project for Software Engineering 2, with the addition of an ESP32-based microcontroller.

Features

- Collects soil moisture, temperature, and humidity data using ESP32-attached sensors
- Transmits sensor data to a Raspberry Pi via Wi-Fi
- Raspberry Pi aggregates, processes, and stores plant data
- Data is visualized in a web-based dashboard
- Modular structure allows adding additional sensors in the future

Project Structure

The sidebar on the right contains the following information:

- About:** A brief description of the project and its purpose.
- Releases:** No releases published. [Create a new release](#)
- Packages:** No packages published. [Publish your first package](#)
- Contributors:** 1 contributor: [nolanruttle](#) (Nolan Tuttle)
- Languages:** (Empty list)

11. Conclusion

Building a modular codebase for the PhytoPi-ESP32 project allowed the team to handle different functions in their own .c files, enforcing separation of concerns and allowing for easier iterative changes compared to a monolithic project structure.

The ESP32 is programmed to react to events instead of constant polling, which is crucial in embedded applications, from saving battery power in some contexts, improving responsiveness, and allowing scalability for multiple subscribers per publisher. This also highlights an important part of the communication protocol, since we are utilizing MQTT with the Raspberry Pi 5 as a publisher. Neither the ESP32 nor the Pi is dependent on another device, which allows for this aforementioned scalability. It also offers persistent connection versus HTTP, making it a much better communication protocol for this use case.

Handling WiFi communication is also important for this project, graceful disconnects, creating methods that wait for a stable connection, and error handling in the wifi manager initialization are all crucial for a WiFi-based IoT application because good testing environments often do not correlate with real-world conditions.

To conclude, this iteration of our ESP32 environmental monitoring project helped us to learn more about modular architecture, good ESP32 and embedded systems coding practices, and handling WiFi protocols properly.

12. References